

Student Setup Guide — Running the App with Docker

This guide gets the whole application — the FastAPI backend and the React web UI — running on your own machine with Docker, and then serves as a **complete Docker command reference** you can keep beside you all term.

How to use it: start with **Quick Start** below to get running in three commands. Everything after that is a categorised reference — build, run, inspect, logs, volumes, cleanup, and so on — so when you need a command, you can find it with its explanation.

A note on safety. Most commands here are harmless and only *show* you things. A few **delete data or images** — they are always called out in the comment beside them (look for words like *wipe*, *nuke*, *DESTRUCTIVE*, or the `-v / --volumes / prune / rm -f` flags). Read the comment before you run one of those.

Quick Start

```
docker compose up --build -d      # build the images and start
everything (detached)
docker compose ps                 # wait until the api row shows "Up
(healthy)"
# → open the app at http://localhost:8080
```

When you are finished for the day:

```
docker compose down              # stop and remove the containers
(your data is kept)
```

That is all you need to run the app. The rest of this guide explains every command in detail.

Prerequisites

```
docker --version                # Docker Engine present
docker compose version          # Compose v2 plugin present
```

Create a `.env` in the project root with the keys below (use Supabase port **6543**, not 5432):

```
SUPABASE_DB_URL=postgresql://postgres:<project-ref>:<password>@aws-0-
<region>.pooler.supabase.com:6543/postgres
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_ANON_KEY=
QDRANT_URL=
QDRANT_API_KEY=
GROQ_API_KEY=
OPENROUTER_API_KEY=
OPENAI_API_KEY=
TAVILY_API_KEY=                # optional
```

```
LANGFUSE_PUBLIC_KEY=      # optional
LANGFUSE_SECRET_KEY=      # optional
LANGFUSE_PROMPTS=false
PROVIDER=openai
```

How to run

```
docker compose up --build -d          # build + start both detached
# → API: http://localhost:8000
# → Web: http://localhost:8080
```

```
docker compose logs -f api            # tail api logs
docker compose down                   # stop + remove containers
```

First boot: ~3–5 min (image build + lifespan warmup). Subsequent boots: ~30 s.

Build

```
docker compose build                  # build both
images
docker compose build api              # build only api
docker compose build web              # build only web
docker compose build --no-cache api   # ignore layer
cache
docker compose build --pull           # pull fresh base
images first
docker compose build --progress=plain api # full build log
(no spinner)
```

```
docker build -f docker/api/Dockerfile -t nawaloka-api:dev . # raw
build
docker build --target builder -f docker/api/Dockerfile .    # stop
at builder stage
docker build --build-arg KEY=value -f docker/api/Dockerfile . # pass
build arg
```

Start

```
docker compose up                    # foreground, both
services
docker compose up -d                 # detached, no
rebuild
docker compose up --build -d         # build + detached
docker compose up -d --build api     # rebuild +
recreate just api
docker compose up -d --no-deps web   # start web
without touching api
```

```
docker compose up -d --force-recreate      # recreate even if
config unchanged                           # also remove
docker compose up -d --remove-orphans      # not used here,
containers not in compose                  but available
docker compose up -d --scale web=2
```

```
docker compose start                       # start existing
stopped containers                         # start one
docker compose start api
```

Stop / kill / pause

```
docker compose stop                       # graceful stop,
keep containers                           # stop one service
docker compose stop api                   # 30s grace period
docker compose stop -t 30                 (default 10s)
```

```
docker compose kill                       # SIGKILL both
(immediate)                               # SIGKILL one
docker compose kill api                   # send a specific
docker compose kill -s SIGTERM api        signal
```

```
docker compose pause api                  # SIGSTOP – freeze
the process                               # SIGCONT – resume
docker compose unpause api
```

```
docker compose restart                    # restart both in
place                                     # restart one (no
docker compose restart api                 rebuild, no env reload)
docker compose restart -t 30 api           # 30s grace before
SIGKILL
```

Down (stop + remove containers)

```
docker compose down                       # stop + remove
containers (keep volumes, keep images)    # ↑ + wipe named
docker compose down -v                   # ↑ + delete
volumes (forces re-download of HF model) # ↑ + delete all
docker compose down --rmi local           # also kill
images built by this compose              # 30s grace before
docker compose down --rmi all              SIGKILL
images referenced (including base images)
docker compose down --remove-orphans
containers not in compose anymore
docker compose down -t 30
```

Inspect

```
docker compose ps                # state + health
of services
docker compose ps -a             # include stopped
docker compose ps --services     # just service
names
docker compose top               # processes inside
each container
docker compose config            # render the
resolved compose file
docker compose config --quiet    # syntax-check only
docker compose port api 8000    # show host port
mapping for a service
docker compose images            # images used by
services

docker ps                        # all running
containers (any source)
docker ps -a                    # include stopped
docker ps -aq                   # just IDs (useful
for piping)
docker ps --filter status=exited # only exited
containers
docker ps --filter name=nawaloka # filter by name
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
```

Logs

```
docker compose logs              # both services,
all-time
docker compose logs api          # one service
docker compose logs -f api       # follow live
docker compose logs --tail=100 api # last 100 lines
docker compose logs --tail=0 -f api # follow only new
lines
docker compose logs --since 2m api # last 2 minutes
docker compose logs --since 2026-04-26T09:00:00 api # since a
timestamp
docker compose logs --until 1m api # up to 1 minute
ago
docker compose logs -t api       # with timestamps
docker compose logs api | grep ROUTER # filter

docker logs nawaloka-api         # raw docker form
docker logs -f --tail=50 nawaloka-api # follow last 50
```

Exec inside a container

```
docker compose exec api bash    # interactive shell
docker compose exec -T api bash -c 'echo hi' # non-interactive
(no TTY)
```

```

docker compose exec -u root api bash                # as a specific
user
docker compose exec -e F00=bar api env                # with extra env
docker compose exec -w /app api ls                    # set working dir
docker compose exec api curl -s http://localhost:8000/health
docker compose exec api env | sort                    # show resolved
env
docker compose exec api python -c 'import torch;
print(torch.__version__)'

docker compose exec web sh                            # shell inside web
(alpine)
docker compose exec web nginx -t                      # validate nginx
config

docker compose run --rm api python scripts/foo.py    # one-off task in a
fresh container
docker compose run --rm --entrypoint sh api           # override
entrypoint

```

Container management (raw docker)

```

docker start nawaloka-api                            # start an existing
stopped container
docker stop nawaloka-api                             # graceful stop
docker stop -t 30 nawaloka-api                       # 30s grace
docker kill nawaloka-api                             # SIGKILL
immediately
docker kill -s SIGTERM nawaloka-api                  # specific signal
docker pause nawaloka-api                            # freeze
docker unpause nawaloka-api                          # resume
docker restart nawaloka-api                          # stop + start
docker rename nawaloka-api nawaloka-api-old          # rename
docker rm nawaloka-api                               # remove stopped
container
docker rm -f nawaloka-api                            # force remove
(kills first)
docker rm $(docker ps -aq)                           # remove ALL
stopped containers
docker rm -f $(docker ps -aq)                        # nuke everything
(running too)
docker wait nawaloka-api                             # block until it
exits, print exit code

```

Inspect deeply

```

docker inspect nawaloka-api
# full JSON
docker inspect --format '{{.State.Status}}' nawaloka-api
# just status
docker inspect --format '{{.State.Health.Status}}' nawaloka-api
# health

```

```

docker inspect --format '{{.NetworkSettings.IPAddress}}' nawaloka-api
# IP
docker inspect --format '{{json .Config.Env}}' nawaloka-api | jq
# env vars
docker inspect --format '{{range .Mounts}}{{.Source}} ->
{{.Destination}}{{"\n"}}{{end}}' nawaloka-api

```

```

docker diff nawaloka-api # files changed
since image was built
docker top nawaloka-api # processes (like
ps)
docker stats # live CPU / RAM /
IO for all
docker stats nawaloka-api nawaloka-web # specific
containers
docker stats --no-stream # one snapshot,
then exit
docker port nawaloka-api # published ports
docker events # live event
stream (start/stop/oom etc.)
docker events --since 10m --until 0s # event history
window

```

Healthcheck

```

docker inspect --format '{{json .State.Health}}' nawaloka-api | jq
docker inspect --format '{{range .State.Health.Log}}{{.Output}}
{{end}}' nawaloka-api

```

Images

```

docker images # all local images
docker images | grep nawaloka # project images
docker images -a # include
intermediate layers
docker image ls --format '{{.Repository}}:{{.Tag}}\t{{.Size}}'
docker image inspect nawaloka-api:latest # full image config
docker image history nawaloka-api:latest # layer history
(size + cmd)

```

```

docker tag nawaloka-api:latest nawaloka-api:v1 # add a tag
docker rmi nawaloka-api:v1 # remove a
tag/image
docker rmi -f nawaloka-api:latest # force (even if
used by stopped containers)
docker image prune # remove dangling
(untagged) images
docker image prune -a # remove ALL
unused images

```

```

docker save nawaloka-api:latest -o api.tar # export image to
tar

```

```
docker load -i api.tar # import image
from tar
```

```
docker pull python:3.13-slim # pull a base
image
```

```
docker push myregistry/nawaloka-api:v1 # push to a
registry (needs login)
```

```
docker login # log in to a
registry
```

```
docker logout
```

Volumes

```
docker volume ls # list all volumes
```

```
docker volume ls | grep nawaloka_hf_cache # project volume
```

```
docker volume inspect nawaloka_hf_cache # mountpoint,
labels
```

```
docker volume create my-volume # create manually
```

```
docker volume rm nawaloka_hf_cache # wipe (forces HF
```

```
model re-download)
```

```
docker volume rm $(docker volume ls -q) # nuke all volumes
```

```
docker volume prune # remove unused
volumes
```

Backup a volume

```
docker run --rm -v nawaloka_hf_cache:/data -v $(pwd):/backup \
    alpine tar czf /backup/hf_cache.tgz -C /data .
```

Restore a volume

```
docker run --rm -v nawaloka_hf_cache:/data -v $(pwd):/backup \
    alpine sh -c "cd /data && tar xzf /backup/hf_cache.tgz"
```

Networks

```
docker network ls # all networks
```

```
docker network ls | grep appcontainerization # project network
```

```
docker network inspect appcontainerization_default # full config
```

```
docker network create my-net # create one
```

```
docker network rm my-net # remove
```

```
docker network prune # remove unused
```

```
docker network connect my-net nawaloka-api # attach a
```

```
container
```

```
docker network disconnect my-net nawaloka-api # detach
```

File copy

```
docker cp nawaloka-api:/app/src/api/main.py . # container → host
```

```
docker cp ./local.py nawaloka-api:/tmp/local.py # host → container
```

```
docker cp - nawaloka-api:/tmp < archive.tar # tar stream →
```

```
container
```

Cleanup

<code>docker compose rm -f</code>	<code># remove stopped</code>
<code>compose containers</code>	
<code>docker container prune</code>	<code># remove all</code>
<code>stopped containers</code>	
<code>docker image prune -a</code>	<code># remove all</code>
<code>unused images</code>	
<code>docker volume prune</code>	<code># remove all</code>
<code>unused volumes</code>	
<code>docker network prune</code>	<code># remove all</code>
<code>unused networks</code>	
<code>docker builder prune</code>	<code># remove build</code>
<code>cache</code>	
<code>docker builder prune -af</code>	<code># ↑ all of it, no</code>
<code>prompt</code>	
<code>docker system df</code>	<code># disk usage by</code>
<code>Docker</code>	
<code>docker system df -v</code>	<code># verbose</code>
<code>breakdown per object</code>	
<code>docker system prune</code>	<code># remove all</code>
<code>dangling everything</code>	
<code>docker system prune -a</code>	<code># ↑ + ALL unused</code>
<code>images</code>	
<code>docker system prune --volumes</code>	<code># ↑ + volumes</code>
<code>(DESTRUCTIVE)</code>	
<code>docker system prune -af --volumes</code>	<code># nuclear option</code>

System info

<code>docker info</code>	<code># daemon info,</code>
<code>storage driver, etc.</code>	
<code>docker version</code>	<code># client + server</code>
<code>versions</code>	
<code>docker context ls</code>	<code># available</code>
<code>contexts (e.g. desktop-linux)</code>	
<code>docker context use desktop-linux</code>	<code># switch context</code>

Common issues

Symptom

port is already allocated on 8000 or 8080

api restarts every ~90 s

could not translate host name aws-0-<region>...

Web shows blank or 502 from /api

First boot >2 min

Code change in src/ not reflected

UI change not reflected
Stuck container won't stop
"no space left on device"

End-to-end flow

1. Create .env (one time) with the keys listed above
`nano .env` *# or your editor of choice*

2. Build + start
`docker compose up --build -d`

3. Wait for api to become healthy
`docker compose ps`
wait until api row shows "Up X (healthy)"

4. Browse to <http://localhost:8080>

5. Tail logs while testing
`docker compose logs -f api`

6. Stop when done
`docker compose down`
or wipe everything including the model cache:
`docker compose down -v`